

Code:

Server.java:

```
/* Brute-Force Login Attack Detection System - SERVER  
    Salted SHA-256 password check + per-IP rate limiting + account logout. */  
import java.io.*;  
import java.net.*;  
import java.security.*;  
import java.time.*;  
import java.time.format.DateTimeFormatter;  
import java.util.*;  
import java.util.concurrent.*;  
  
public class Server {  
    static final int PORT = 6001;  
    static final long WINDOW_MS = 10_000;    // sliding window  
    static final int MAX_FAILS = 5;          // fails allowed in window  
    static final long BLOCK_MS = 30_000;     // block duration  
    static final long RUN_MS = 20_000;       // demo auto-stop  
  
    static Map<String, byte[]> salts = new HashMap<>();  
    static Map<String, byte[]> hashes = new HashMap<>();  
    static Map<String, Deque<Long>> failLog = new ConcurrentHashMap<>();  
    static Map<String, Long> blocked = new ConcurrentHashMap<>();  
    static Set<String> lockedAccounts = ConcurrentHashMap.newKeySet();  
  
    static void addUser(String user, String pass) throws Exception {  
        byte[] salt = new byte[16];  
        new SecureRandom().nextBytes(salt);
```

```
    salts.put(user, salt);  
    hashes.put(user, sha256(salt, pass));  
}
```

```
static byte[] sha256(byte[] salt, String pass) throws Exception {  
    MessageDigest md = MessageDigest.getInstance("SHA-256");  
    md.update(salt);  
    return md.digest(pass.getBytes());  
}
```

```
static boolean checkPassword(String user, String pass) throws Exception {  
    if (!hashes.containsKey(user)) return false;  
    return Arrays.equals(hashes.get(user), sha256(salts.get(user), pass));  
}
```

```
static boolean isBlocked(String ip) {  
    Long t = blocked.get(ip);  
    if (t == null) return false;  
    if (System.currentTimeMillis() < t) return true;  
    blocked.remove(ip);  
    return false;  
}
```

```
static boolean recordFailure(String ip) {  
    long now = System.currentTimeMillis();  
    Deque<Long> dq = failLog.computeIfAbsent(ip, k -> new ArrayDeque<>());  
    dq.addLast(now);  
    while (!dq.isEmpty() && now - dq.peekFirst() > WINDOW_MS) dq.pollFirst();  
}
```

```

        if (dq.size() >= MAX_FAILS) {

            blocked.put(ip, now + BLOCK_MS);

            System.out.printf("[!] ALERT: %s made %d failed attempts in %ds -> BLOCKED
for %ds%n",

                ip, dq.size(), WINDOW_MS / 1000, BLOCK_MS / 1000);

            return true;

        }

        return false;

    }

    public static void main(String[] args) throws Exception {

        System.out.println("=" .repeat(62));

        System.out.println(" BRUTE-FORCE LOGIN ATTACK DETECTION AND PREVENTION
SYSTEM - SERVER");

        System.out.println("=" .repeat(62));

        addUser("admin", "P@ssw0rd!2024");

        addUser("suruthi", "SecureNet#99");

        System.out.println("[*] Registered users: " + hashes.keySet());

        System.out.printf("[*] Policy: block an IP after %d failed logins in %ds, for %ds%n",

            MAX_FAILS, WINDOW_MS / 1000, BLOCK_MS / 1000);

        ServerSocket srv = new ServerSocket(PORT);

        srv.setSoTimeout(1000);

        System.out.println("[*] Listening on 127.0.0.1:" + PORT + " ...");

        long deadline = System.currentTimeMillis() + RUN_MS;

        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("HH:mm:ss");

```

```

while (System.currentTimeMillis() < deadline) {
    Socket conn;

    try { conn = srv.accept(); } catch (SocketTimeoutException e) { continue; }

    String ip = conn.getInetAddress().getHostAddress();

    try (DataInputStream in = new DataInputStream(conn.getInputStream());
        DataOutputStream out = new DataOutputStream(conn.getOutputStream())) {

        String username = readString(in);

        String password = readString(in);

        if (isBlocked(ip)) {

            System.out.printf("[X] %12s -> login '%s' REJECTED (IP currently blocked)%n",
ip, username);

            out.writeUTF("BLOCKED"); continue;

        }

        if (lockedAccounts.contains(username)) {

            System.out.printf("[X] %12s -> login '%s' REJECTED (account locked)%n", ip,
username);

            out.writeUTF("LOCKED"); continue;

        }

        boolean ok = checkPassword(username, password);

        String ts = LocalTime.now().format(fmt);

        if (ok) {

            System.out.printf("[+] %s %12s -> login '%s' SUCCESS%n", ts, ip, username);

            failLog.getOrDefault(ip, new ArrayDeque<>()).clear();

            out.writeUTF("OK");

        } else {

            System.out.printf("[-] %s %12s -> login '%s' FAILED (wrong password)%n", ts,
ip, username);

            boolean blockedNow = recordFailure(ip);

```

```

        if (blockedNow) { lockedAccounts.add(username);
out.writeUTF("BLOCKED"); }

        else out.writeUTF("FAIL");

    }

    } catch (Exception e) {

        // ignore malformed/closed connections

    } finally {

        conn.close();

    }

}

System.out.println();

System.out.println("[*] Server shutting down (demo window ended)");

System.out.println("[*] Summary: blocked IPs = " + blocked.keySet() + ", locked
accounts = " + lockedAccounts);

    srv.close();

}

```

```

static String readString(DataInputStream in) throws IOException {

    int len = in.readUnsignedShort();

    byte[] b = new byte[len];

    in.readFully(b);

    return new String(b);

}

}

```

LegitUser.java:

```

/* Simulates a legitimate user logging in with the correct password. /import java.io.;
import java.net.*;

```

```

public class LegitUser { static final String HOST = "127.0.0.1"; static final int PORT =
6001;

static String attempt(String user, String pass) throws IOException {
    try (Socket s = new Socket()) {
        s.connect(new InetSocketAddress(HOST, PORT), 3000);
        DataOutputStream out = new
DataOutputStream(s.getOutputStream());
        byte[] ub = user.getBytes(); out.writeShort(ub.length);
out.write(ub);
        byte[] pb = pass.getBytes(); out.writeShort(pb.length);
out.write(pb);
        DataInputStream in = new
DataInputStream(s.getInputStream());
        return in.readUTF();
    }
}

public static void main(String[] args) throws Exception {
    System.out.println("=".repeat(62));
    System.out.println("    LEGITIMATE USER - login with correct
credentials");
    System.out.println("=".repeat(62));
    String r = attempt("suruthi", "SecureNet#99");
    System.out.println("[*] login 'suruthi' with correct password ->
" + r);
}

}

```

Attacker.java:

```

/* Brute-Force Login Attack Detection System - ATTACKER (demo script) / import
java.io.; import java.net.*;

public class Attacker { static final String HOST = "127.0.0.1"; static final int PORT =
6001; static final String USERNAME = "admin"; static final String[] PASSWORD_LIST =
{"123456", "password", "admin123", "letmein", "qwerty", "football", "P@ssword1",
"welcome1", "monkey123", "iloveyou"};

static String attempt(String user, String pass) throws IOException {
    try (Socket s = new Socket()) {

```

```

        s.connect(new InetSocketAddress(HOST, PORT), 3000);
        DataOutputStream out = new
DataOutputStream(s.getOutputStream());
        writeString(out, user);
        writeString(out, pass);
        DataInputStream in = new
DataInputStream(s.getInputStream());
        return in.readUTF();
    }
}

static void writeString(DataOutputStream out, String s) throws
IOException {
    byte[] b = s.getBytes();
    out.writeShort(b.length);
    out.write(b);
}

public static void main(String[] args) throws Exception {
    System.out.println("=".repeat(62));
    System.out.println("    BRUTE-FORCE LOGIN ATTACK DETECTION AND
PREVENTION SYSTEM - ATTACKER");
    System.out.println("=".repeat(62));
    System.out.printf("[*] Target user: '%s' | password list
size: %d\n", USERNAME, PASSWORD_LIST.length);
    int i = 1;
    for (String pw : PASSWORD_LIST) {
        String result;
        try { result = attempt(USERNAME, pw); }
        catch (IOException e) { System.out.println("[X] Server not
reachable"); return; }
        System.out.printf("[%02d] trying password '%s': -> %s\n",
i++, pw, result);
        if (result.equals("BLOCKED")) {
            System.out.println("[!] Attacker IP has been BLOCKED by
the server - attack stopped");
            return;
        }
        Thread.sleep(300);
    }
    System.out.println("[*] Password list exhausted without
success");
}

```

}

}

Output:

Command Prompt

```
C:\acer\document\CNS\Miniproject>java Server
[*] Listening on 127.0.0.1:6001 ...
[-] 04:12:11 127.0.0.1 -> login 'admin' FAILED (wrong password)
[-] 04:12:11 127.0.0.1 -> login 'admin' FAILED (wrong password)
[-] 04:12:11 127.0.0.1 -> login 'admin' FAILED (wrong password)
[-] 04:12:12 127.0.0.1 -> login 'admin' FAILED (wrong password)
[-] 04:12:12 127.0.0.1 -> login 'admin' FAILED (wrong password)
[!] ALERT: 127.0.0.1 made 5 failed attempts in 10s -> BLOCKED for 30s
[X] 127.0.0.1 -> login 'suruthi' REJECTED (IP currently blocked)

[*] Server shutting down (demo window ended)
[*] Summary: blocked IPs = [127.0.0.1], locked accounts = [admin]

C:\acer\document\CNS\Miniproject>
```

Command Prompt

```
C:\acer\document\CNS\Miniproject>java Attacker
=====
BRUTE-FORCE LOGIN ATTACK DETECTION AND PREVENTION SYSTEM - ATTACKER
=====
[*] Target user: 'admin' | password list size: 10
[01] trying password '123456': -> FAIL
[02] trying password 'password': -> FAIL
[03] trying password 'admin123': -> FAIL
[04] trying password 'letmein': -> FAIL
[05] trying password 'qwerty': -> BLOCKED
[!] Attacker IP has been BLOCKED by the server - attack stopped

C:\acer\document\CNS\Miniproject>
```

Command Prompt

```
C:\acer\document\CNS\Miniproject>java LegitUser
=====
LEGITIMATE USER - login with correct credentials
=====
[*] login 'suruthi' with correct password -> BLOCKED

C:\acer\document\CNS\Miniproject>
```